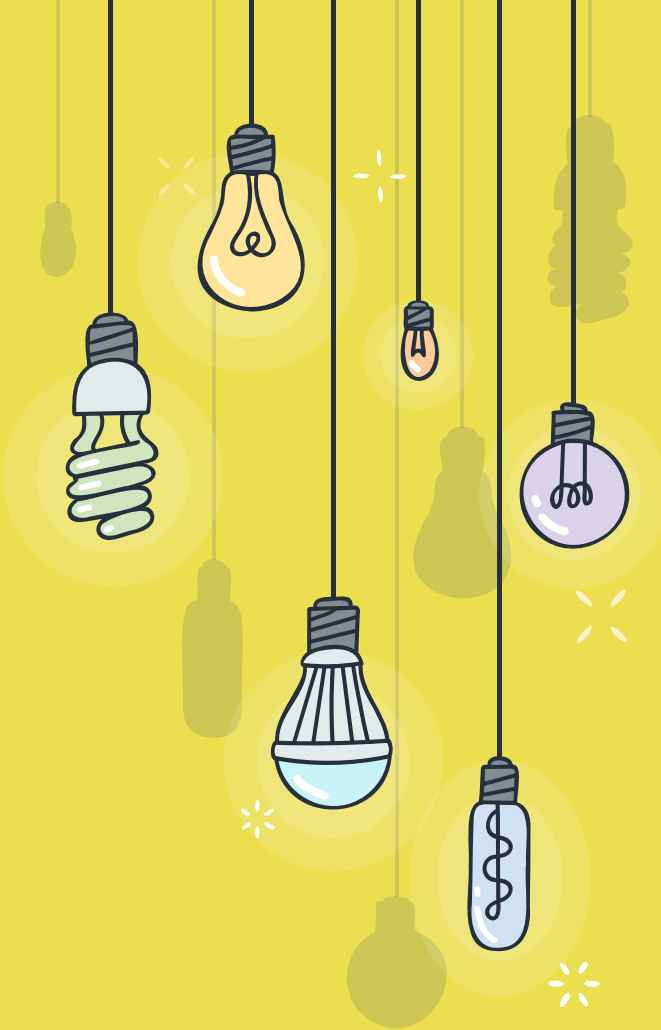


PYTHON

김 지 원

DATA STUCTURE



조합 데이터 타입

- 순서형 자료형(sequence data type)
  - 문자열, 튜플, 리스트
- 비순서형 자료형(non-sequential data type)
  - 세트, 딕셔너리

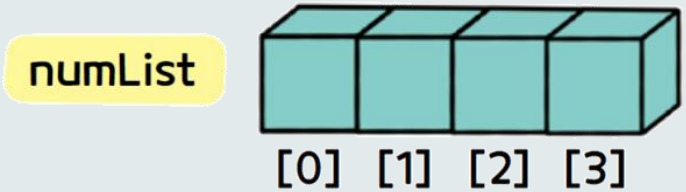
이름	특징
리스트 : [ ]	변경가능한 자료형, 순서형 자료. 메모리 크기가 크다
tuple : ( )	변경불가한 자료형, 읽기 전용. 순서형 자료, 메모리 크기 작다
set : { }	변경가능 자료형, 비순서형 자료. 데이터 중복 안됨.
dictionary : { }	변경가능 자료형. 비순서형 자료. 메모리 크기가 매우 크다. 키와 값을 동시에 저장, key는 변경불가

1 LIST



LIST의 개념

- 하나씩 사용하던 변수를 붙여서 한 줄로 붙여놓은 개념
- 리스트는 전체의 이름(numList)을 지정하여 사용
- 각각의 데이터에는 번호(첨자)를 붙여서 접근함



LIST 생성

- 대괄호 안에 값을 선언함

```
리스트 이름 = [값1, 값2, 값3, … ]
```

```
numList = [10, 20, 30, 40]
numList[0]
numList[1]
numList[2]
numList[3]
```

LIST의 덧셈과 곱셈

- 리스트의 덧셈
  - 요소들이 합쳐져 하나의 리스트가 됨
- 리스트의 곱셈
  - 곱한 횟수만큼 리스트가 반복됨

```
numList = [10, 20, 30]
myList = [40, 50, 60]

print(numList + myList)
[10, 20, 30, 40, 50, 60]

print(numList * 3)
[10, 20, 30, 10, 20, 30, 10, 20, 30]
```

LIST METHOD

메소드	기능
리스트.index()	리스트에 데이터의 위치(index번호)
리스트.append()	리스트의 끝에 데이터를 추가
리스트.insert()	리스트의 특정 위치에 데이터를 삽입
리스트.remove()	리스트의 처음 나오는 특정 데이터를 삭제
리스트.clear()	리스트의 모든 데이터를 삭제
리스트.sort ()	오름차순으로 정렬(내림차순 정렬 : reverse=True)
리스트.reverse()	리스트의 값을 역순으로 뒤집기
리스트.pop()	리스트의 맨 마지막 요소를 돌려주고, 그 요소는 삭제
리스트.count()	리스트 내에 특정 값이 몇 개 있는지 알려줌
리스트.extend([값])	리스트의 뒤에 값 추가

내장함수

내장함수	기능
in not in	리스트에 해당 값이 있는지 검사. 문자열, 튜플, 딕셔너리, 리스트에 사용
del 리스트[위치] del(리스트[위치])	리스트의 특정 위치의 데이터 삭제
len(리스트)	리스트의 길이
sorted(리스트)	오름차순으로 정렬하여 리스트 형태로 출력(내림차순 정렬 reverse=True). 원본의 순서는 변경되지 않는다.
sum(리스트)	숫자 데이터로 된 리스트의 합을 구함
max(리스트)	숫자 데이터로 된 리스트의 최대값을 구함
enumerate(리스트)	반복문 사용시 데이터 뿐만 아니라 인덱스(index)도 함께 출력하고 싶을 때 사용. tuple형태로 반환
zip(리스트, 리스트)	리스트 여러개를 동일한 자리끼리 묶어줌

2차원 LIST

- 1차원 리스트를 여러 개 연결한 리스트
- 2개의 첨자를 사용함
  - 리스트이름[행][열]

```
numList2 = [ [1, 2, 3, 4],
             [5, 6, 7, 8],
             [9, 10, 11, 12] ]
```



중첩 FOR문

```
numList2 = [[1, 2, 3, 4],
            [5, 6, 7, 8],
            [9, 10, 11, 12]]

for i in range(0, 3) :
    for k in range(0, 4) :
        print(" ", numList2[i][k], end='')
    print("")
```

[실행결과]

```
 1  2  3  4
 5  6  7  8
 9 10 11 12
```

2 TUPLE



TUPLE

- 읽기 전용의 리스트
- 소괄호로 생성
- 값을 변경할 수 없는 것만 제외하면 리스트와 동일

- 튜플 생성
  - 튜플은 소괄호로 생성하지만, 괄호가 없어도 무방함
  - 튜플의 항목이 한 개일 때 튜플 뒤에 콤마(,)를 붙여야 함

```
tup1 = (10, 20, 30)
tup2 = 10, 20, 30
tup3 = (10,)
tup4 = 10,
tup5 = (10)  # int형
tup6 = 10    # int형
```

TUPLE

- 읽기 전용인 튜플
- 다음은 모두 오류가 발생함

```
numTup1.append(40)
numTup1[0] = 40
del(numTup1[0])
```

- 튜플 자체를 통째로 삭제하려면 del(튜플이름) 함수를 사용함

```
del(numTup1)
del(numTup2)
```

TUPLE

- 튜플에 접근하기
- 튜플이름[인덱스] : 특정 항목에 접근하기

```
numTup1 = (10, 20, 30, 40)
print(numTup1[0])
print(numTup1[0] + numTup1[1] + numTup1[2])
```

- 인덱싱사용: 튜플의 범위에 접근하기

```
print(numTup1[1:3])
(20, 30)
print(numTup1[1:])
(20, 30, 40)
print(numTup1[:3])
(10, 20, 30)
```

TUPLE의 덧셈과 곱셈

```
numTup1 = (10, 20, 30, 40)
numTup2 = ('A', 'B')
print(numTup1 + numTup2)
(10, 20, 30, 40, 'A', 'B')
print(numTup2 * 3)
('A', 'B', 'A', 'B', 'A', 'B')
```

TUPLE의 메소드와 내장함수

메소드	기능
튜플.count(데이터)	해당 튜플에 지정한 데이터의 개수를 반환
튜플.index(value [, start .index])	value로 지정된 값의 Index번호 반환 start_index를 지정하면 해당 index 부터 value를 찾음 없으면 에러 발생

내장함수	기능
len()	튜플에 저장된 값의 개수를 반환
max()	튜플에 저장된 값중에서 최대값 반환
min()	튜플에 저장된 값중에서 최소값 반환
sum()	튜플에 저장된 값들의 합계 반환
sorted()	튜플에 저장된 값들을 오름차순으로 정렬하여 리스트 형태로 출력 (내림차순 정렬 : reverse=True) 원본의 순서는 변경되지 않는다

3 DICTIONARY



DICTIONARY

- 2개의 쌍이 하나로 묶이는 자료구조를 의미
  - 키(Key)와 값(Value)의 쌍으로 이루어짐
- 중괄호{}로 묶여 있음

```
딕셔너리변수 = { 키1:값1 , 키2:값2, 키3:값3, … }
```

- 딕셔너리 생성하기
  - 키와 값을 사용자가 지정 즉 자료형과는 관련 없음
  - 순서가 없음

```
myDict = {1:'a', 2:'b', 3:'c'}
myDict = {'a':1, 'b':2, 'c':3}
```

DICTIONARY

- 여러 개의 정보를 하나의 변수로 표현할 때 유용하게 사용됨
- 회사원 홍길동의 정보를 딕셔너리로 생성하기

회사원의 정보

키(Key)	값(Value)
사번	1000
이름	홍길동
부서	그린컴퓨터

```
empDict = {'사번':1000 , '이름':'홍길동', '부서':'그린컴퓨터'}
{'사번': 1000, '이름': '홍길동', '부서':  ' 그린컴퓨터'}
```

DICTIONARY

- 딕셔너리에 정보를 더 추가하기
  - 정보를 추가할 때는 키와 값을 쌍으로 추가
  - ‘딕셔너리이름[키] = 값’ 형식 사용

```
empDict['연락처'] = '010-111-2222'
{'사번': 1000, '이름': '홍길동', '부서': '그린컴퓨터', '연락처': '010-111-2222'}
```

- 키가 딕셔너리에 존재하는 경우
  - 기존의 키에 대한 값이 변경됨

```
empDict['이름'] = '아무개'
{'사번': 1000, '이름': '아무개', '부서': ' 그린컴퓨터', '연락처': '010-111-2222'}
```

DICTIONARY

- 딕셔너리의 키는 유일함
  - 만약 동일한 키를 갖는 딕셔너리를 생성하면,
    마지막에 있는 키의 값이 적용되어 생성됨

```
empDict2 = {'사번':1000 , '이름':'홍길동', '사번':2000}
{'사번': 2000, '이름': '홍길동'}
```

DICTIONARY의 메소드

메소드	기능
딕셔너리.keys()	딕셔너리의 모든키들을 dict_keys로 가져온다
딕셔너리.values()	딕셔너리의 모든값들을 dict_values로 가져온다.
딕셔너리.items()	딕셔너리의 키와 값을 dict_items로 가져온다.
딕셔너리.get(키)	딕셔너리의 키로 값을 가져온다.
딕셔너리.clear()	딕셔너리의 모든 데이터 삭제
딕셔너리.update()	딕셔너리에 딕셔너리 데이터 추가

DICTIONARY의 내장함수

내장함수	기능
in, not in	딕셔너리에 해당 key가 있는지 검사
del	딕셔너리의 특정 데이터 삭제
len(딕셔너리)	딕셔너리에 저장된 값의 개수 반환
sorted(딕셔너리)	오름차순으로 정렬하여 리스트 형태로 출력 (내림차순 정렬 reverse =True). 원본의 순서는 변경되지 않음

4

SET



SET

- 파이썬 2.3부터 지원하기 시작한 자료형
- 집합에 관련된 것을 쉽게 처리하기 위해 만든 자료형
- 중복을 허용하지 않는다.
  - 자료형의 중복을 제거하기 위한 필터 역할로 종종 사용하기도 함
- 순서가 없다(Unordered) : 그러므로 인덱싱 사용못함
- 중괄호 {} 사용

```
s1 = {3, 5, 7}
{3, 5, 7}
s2 = {1, 2, 3, 1, 2, 3, 4}
{1, 2, 3, 4}
```

SET의 메소드

메소드	기능
세트.intersection()	교집합(& 기호 쓴것과 동일)
세트.union()	합집합(| 기호 쓴것과 동일)
세트.defference()	차집합
세트.add()	세트에 요소 추가
세트.pop()	세트에서 임의의 요소를 삭제하고 해당 요소를 반환. 만약 요소가 없으면 에러를 발생
세트.clear()	세트의 모든 데이터를 삭제
세트.remove()	세트에서 특정 요소를 삭제하고 요소가 없으면 에러 발생
세트.discard()	세트에서 특정 요소를 삭제하고 요소가 없으면 그냥 넘어감

DICTIONARY의 내장함수

내장함수	기능
in, not in	세트에 해당 값이 있는지 검사
len(세트)	세트에 저장된 값의 개수 반환

5 TYPE 변환



TYPE 변환

- list(), tuple(), set() 함수를 이용하여 type 변환

Thanks!

- Any questions?

